AI in Industry

Autonomous and Mobile Robotics - Project

Francesco Cenni Giulio Mazzaroppi Giacomo Pezzolati

francesco.cenni3@studio.unibo.it
giulio.mazzaroppi@studio.unibo.it
giacomo.pezzolati@studio.unibo.it

University of Bologna

Academic Year 2025/2026

February 3, 2026

Overview

1. Introduction
2. Task 1: Map Generation
3. Task 2: Navigation
4. Task 3: Pick and Place

Project Overview

- **Objective:** Develop a complete autonomous system for the Tiago robot combining navigation and manipulation skills.
- **Platform:** Tiago Robot (Simulated in Gazebo) [**?**].
- **Key Tasks:**
  1. Map Generation (SLAM) [**?**].
  2. Autonomous Navigation & Localization [**?**].
  3. Pick and Place of specific ArUco cubes [**?**].

System Architecture

- **ROS2 Version:** Humble (implied by context)
- **Package Structure:**
  - `tiago_exam`: Launch files.
  - `tiago_exam_worlds`: Simulation environments [**?**].
  - `[Your_Package_Name]`: Custom logic nodes.
- **Launch Strategy:** Three distinct launch files created for the three tasks as requested [**?**].

Task 1: Strategy & Architecture

Goal: autonomously explore and map the environment.

- **Mapping Algorithm:** SLAM via `tiago_nav_bringup` (`slam:=True`).
- **Autonomous Exploration:**
  - Utilized `explore_lite` package to drive the robot to unknown areas automatically.
- **Safety Mechanism:**
  - Implemented `tuck_arm` node to ensure the arm is safely folded before navigation starts.

Task 1: Safety Configuration (Tuck Arm)

- **Purpose:** Prevent arm collisions during exploration.
- **Implementation:** Custom ROS2 node sending goals to MoveIt.
- **Code Snippet:**

```
# From tuck_arm.py
self.home_joints = [0.20, -1.34, -0.20, 1.94, -1.57, 1.37,

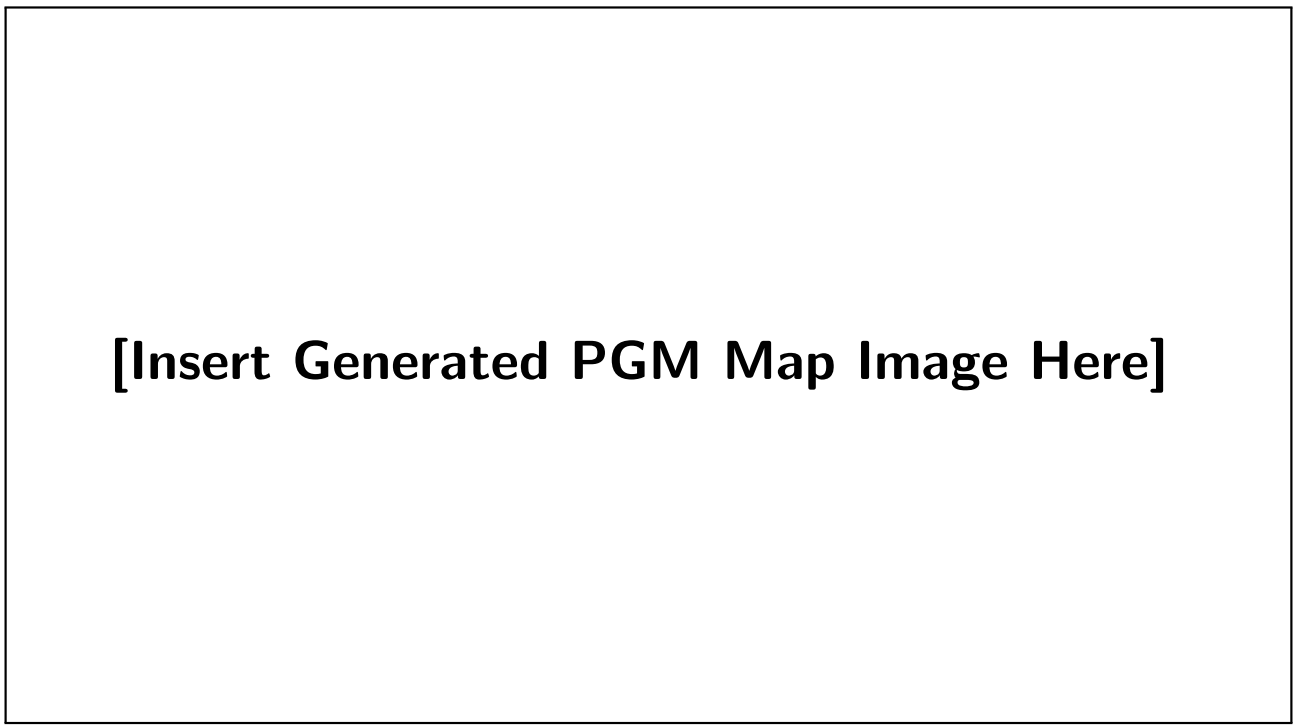
# Sending Goal to MoveIt
goal_msg = MoveGroup.Goal()
goal_msg.request.group_name = 'arm'
goal_msg.request.goal_constraints.append(constraints)
self._action_client.send_goal_async(goal_msg)
```

Task 1: Map Saving Utility

- **Innovation:** Instead of manually running CLI commands, we created a service-based watcher node.
- **Functionality:** Exposes a `save_map` service that triggers the `map_saver_cli` internally.

```
# From map_saver_watcher.py
def save_map_callback(self, request, response):
    cmd = [
        'ros2', 'run', 'nav2_map_server', 'map_saver_cli',
        '-f', self.map_path, '--fmt', 'pgm'
    ]
    subprocess.run(cmd, check=True)
    response.success = True
    return response
```

Task 1: Resulting Map



Map generated autonomously using `explore_lite` and saved via `map_saver_watcher`.

Task 2: Architecture Overview

The navigation system is orchestrated by a central **Coordinator Node** managing a Finite State Machine (FSM).

- **Coordinator:** Handles high-level logic (Exploring $\leftrightarrow$ Tracking) using `ReentrantCallbackGroup` for concurrent service handling.
- **Naive Explorer:** Generates safe random goals using map image analysis.
- **ArUco Tracker:** Performs TF calculations and active head tracking.
- **Navigator:** Wrapper for Nav2 and custom 360° rotation.

Task 2: Global Localization Strategy

Problem: Robot spawns in a random unknown location.

1. **Particle Spread:** Calls `/reinitialize_global_localization` to distribute AMCL particles.
2. **Convergence Loop:** Rotates the robot 360° (1.0 rad/s) to gather sensor data until covariance thresholds are met (< 0.03).

```
# From localization.py
def check_covariance(self):
    if (var_x < 0.03 and var_y < 0.03 and var_yaw < 0.03):
        return True
    return False
```

Task 2: Safe Goal Generation

Instead of hardcoded waypoints, we implemented a **Naive Explorer** using Computer Vision on the map image:

- **Map Parsing:** Loads the YAML/PGM map file.
- **Safety Margin:** Applies `cv2.distanceTransform` to create a "Safety Map".
- **Pixel Selection:** Filters pixels where distance to obstacle $>$ clearance (0.7m).

This ensures generated random goals are never inside or too close to obstacles.

Task 2: Coordinator FSM

The `coordinator_task2.py` implements a State Machine with interrupt capabilities.

- **Exploration:** Robot navigates to random goals.
- **Interrupt:** If `aruco_tracker` detects a new ID, the FSM switches to **TRACKING** immediately.

```
# From coordinator_task2.py
case State.EXPLORING:
    if self.is_new_marker_detected:
        self.state = State.TRACKING
        self.nav_outcome = False
        continue # Interrupt loop
```

Task 2: Visual Servoing & Transforms

To approach the marker accurately, we compute the real pose using KDL chains:

$$T_{map}^{marker} = T_{map}^{base} \cdot T_{base}^{camera} \cdot T_{camera}^{marker}$$

Active Head Tracking: The head automatically centers the marker during approach by calculating angular deltas:

- $\Delta_{pan} = \arctan(x_{raw}/z_{raw})$
- $\Delta_{tilt} = \arctan(y_{raw}/d_{ground})$

Task 2: Data Persistence

Upon successful inspection, marker locations are saved for Task 3.

- **Storage:** CSV file with Timestamp, ID, Position, and Orientation.
- **Visualization:** Broadcasts a `StaticTransform` so markers appear permanently in RViz.

```
# From aruco_noter.py
writer.writerow([timestamp, name, x, y, z, qx, qy, qz, qw])
self.tf_static_broadcaster.sendTransform(t)
```

Task 3: Coordination Strategy

The `coordinator_task3` node manages the full transport cycle.

- **Data Loading:** Loads exact Pick and Place locations saved in Task 2 from `marker_locations.csv`.
- **Sequence:** Enforces the mandatory order: Cube **63** followed by Cube **582**.
- **Visual Feedback:** Publishes `target_cube_frame` to TF for real-time visualization in RViz.

Task 3: MoveIt2 Integration

The `manipulator` node interfaces with MoveIt2 using the `RRTConnectkConfigDefault` planner.

- **Collision Avoidance:** Added virtual collision boxes (IDs 10-13) to prevent the arm from hitting the floor or robot base.
- **Safety:** Implemented a "Safe Joint Configuration" to retract the arm during navigation.

```
# From manipulator.py
self.safe_joints_position = [
    0.0, 0.20, -1.34, -0.20, 1.94, -1.57, 1.37, 0.00
]
```

Task 3: Robust Grasping (Gazebo)

To ensure stability in the simulation (as suggested in the guidelines), we implemented the **Link Attacher** plugin logic.

- **Approach:** The arm moves to a "Pre-Grasp" pose defined by a $-0.4m$ offset along the X-axis of the marker.
- **Attaching:** Once the gripper closes, we call the `/ATTACHLINK` service to rigidly bind the cube to the gripper link.

This prevents the object from slipping during the robot's navigation phases.

Task 3: Placement & Release

The placement pose is calculated dynamically relative to the detected grasp transform.

- **Logic:** We reuse the grasp orientation but apply a vertical offset to ensure the cube is released *on top* of the table.
- **Offset:** $+15cm$ $(0.15m)$ on the Z-axis.

```
# From coordinator_task3.py
release_frame = kdl.Frame(self.base_T_marker)
release_frame.p[2] += 0.15 # Raise Z by 15 cm
```

After placement, the `/DETACHLINK` service is called to release the object physics.